



Base de Données - JobbingTrack

Structure complète de la base de données PostgreSQL de JobbingTrack v4.1.

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)



Vue d'ensemble

Base de données PostgreSQL 15+ avec architecture multi-schémas, relations polymorphes et support mobile/offline.



Architecture

Configuration PostgreSQL

```
# docker-compose.yml
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: jobbingtrack
    POSTGRES_USER: jobbingtrack
    POSTGRES_PASSWORD: jobbingtrack123
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U jobbingtrack -d jobbingtrack"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Principes de conception

- **Multi-schémas** : Isolation par service
 - **Relations polymorphes** : Flexibilité des associations
 - **Historisation** : Suivi des changements
 - **Synchronisation** : Support offline mobile
 - **Audit trail** : Traçabilité complète
-



Modèles principaux

Nouveautés v4.1

ApplicationStatusHistory

Historique des changements de statut des candidatures

```
CREATE TABLE applications.application_status_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  application_id UUID REFERENCES applications.applications(id),
  old_status VARCHAR(50),
  new_status VARCHAR(50) NOT NULL,
  changed_by UUID,
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  notes TEXT
);
```

Notification

Système de notifications multi-canaux

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  type VARCHAR(50) NOT NULL, -- email, push, sms, in_app
  title VARCHAR(255) NOT NULL,
  message TEXT NOT NULL,
  entity_type VARCHAR(50), -- application, interview, etc.
  entity_id UUID,
  is_read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Event

Calendrier avec relations polymorphes

```
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  start_date TIMESTAMP NOT NULL,
  end_date TIMESTAMP,
  event_type VARCHAR(50) NOT NULL,
  entity_type VARCHAR(50), -- application, interview, call, followup
);
```

```
entity_id UUID,
reminder_at TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

SyncQueue

Synchronisation mobile/offline

```
CREATE TABLE sync_queues (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id UUID NOT NULL,
  action VARCHAR(20) NOT NULL, -- create, update, delete
  data JSONB NOT NULL,
  synced BOOLEAN DEFAULT FALSE,
  synced_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Relations many-to-many

Contact ↔ Entreprise

```
CREATE TABLE companies.contact_companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  company_id UUID NOT NULL REFERENCES companies.companies(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(contact_id, company_id)
);
```

Contact ↔ Candidature

```
CREATE TABLE applications.contact_applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contact_id UUID NOT NULL,
  application_id UUID NOT NULL REFERENCES applications.applications(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

    UNIQUE(contact_id, application_id)
);

```

Contact ↔ Relance

```

CREATE TABLE followups.follow_up_contacts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contact_id UUID NOT NULL,
    follow_up_id UUID NOT NULL REFERENCES followups.followups(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(contact_id, follow_up_id)
);

```

Contact ↔ Entretien

```

CREATE TABLE interviews.interview_contacts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contact_id UUID NOT NULL,
    interview_id UUID NOT NULL REFERENCES interviews.interviews(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(contact_id, interview_id)
);

```

Contact ↔ Événement

```

CREATE TABLE events.contact_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contact_id UUID NOT NULL,
    event_id UUID NOT NULL REFERENCES events(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(contact_id, event_id)
);

```



Enums et types

Statuts de candidatures

```
CREATE TYPE application_status AS ENUM (
    'applied', 'screening', 'phone_interview', 'technical_interview',
    'final_interview', 'offer', 'rejected', 'withdrawn', 'archived'
);
```

Types d'événements

```
CREATE TYPE event_type AS ENUM (
    'interview', 'call', 'followup', 'deadline', 'reminder', 'meeting'
);
```

Types de notifications

```
CREATE TYPE notification_type AS ENUM (
    'email', 'push', 'sms', 'in_app'
);
```

Rôles utilisateurs

```
CREATE TYPE user_role AS ENUM (
    'user', 'admin', 'super_admin'
);
```

Indexes et performances

Indexes principaux

```
-- Applications
CREATE INDEX idx_applications_user_id ON applications.applications(user_id);
CREATE INDEX idx_applications_status ON applications.applications(status);
CREATE INDEX idx_applications_company_id ON applications.applications(company_id);
CREATE INDEX idx_applications_created_at ON applications.applications(created_at);

-- Contacts
CREATE INDEX idx_contacts_user_id ON companies.contacts(user_id);
```

```

CREATE INDEX idx_contacts_company_id ON companies.contacts(company_id);
CREATE INDEX idx_contacts_email ON companies.contacts(email);
CREATE INDEX idx_contacts_last_contact ON companies.contacts(last_contact_date);

-- Entretiens
CREATE INDEX idx_interviews_user_id ON interviews.interviews(user_id);
CREATE INDEX idx_interviews_date ON interviews.interviews(scheduled_at);
CREATE INDEX idx_interviews_status ON interviews.interviews(status);

-- Historique
CREATE INDEX idx_application_status_history_application_id ON applications.applications(status_history_application_id);
CREATE INDEX idx_application_status_history_changed_at ON applications.applications(status_history_changed_at);

```

Indexes many-to-many

```

-- Tables de liaison
CREATE INDEX idx_contact_companies_contact_id ON companies.contact_companies(contact_id);
CREATE INDEX idx_contact_companies_company_id ON companies.contact_companies(company_id);
CREATE INDEX idx_contact_applications_contact_id ON applications.contact_applications(contact_id);
CREATE INDEX idx_contact_applications_application_id ON applications.contact_applications(application_id);

```

Optimisations avancées

```

-- Index partiels (pour les données actives)
CREATE INDEX idx_applications_active ON applications.applications(user_id, status)
WHERE is_archived = FALSE;

-- Index composites
CREATE INDEX idx_applications_user_status ON applications.applications(user_id, status);
CREATE INDEX idx_contacts_user_company ON companies.contacts(user_id, company_id);

-- Index GIN pour recherche textuelle
CREATE INDEX idx_companies_search ON companies.companies USING GIN (to_tsvector('english', name));

```



Schémas par service

Auth (authentification)

```

-- Schéma: auth
CREATE SCHEMA auth;

```

```
-- Utilisateurs
CREATE TABLE auth.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role user_role DEFAULT 'user',
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  phone VARCHAR(50),
  profile_picture VARCHAR(500),
  is_active BOOLEAN DEFAULT TRUE,
  reset_token VARCHAR(255),
  reset_token_expiry TIMESTAMP,
  is_deleted BOOLEAN DEFAULT FALSE,
  is_archived BOOLEAN DEFAULT FALSE,
  sync_hash VARCHAR(64),
  entity_hash VARCHAR(64),
  last_sync_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sessions
CREATE TABLE auth.sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  token VARCHAR(500) NOT NULL,
  refresh_token VARCHAR(500),
  expires_at TIMESTAMP NOT NULL,
  ip_address INET,
  user_agent TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Applications (candidatures)

```
-- Schéma: applications
CREATE SCHEMA applications;

-- Candidatures
CREATE TABLE applications.applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  company_id UUID,
  platform_id UUID,
  position VARCHAR(255) NOT NULL,
  description TEXT,
  location VARCHAR(255),
```

```

type VARCHAR(100),
salary_min DECIMAL(10,2),
salary_max DECIMAL(10,2),
status application_status DEFAULT 'applied',
application_date DATE,
job_url VARCHAR(500),
notes TEXT,
is_archived BOOLEAN DEFAULT FALSE,
archived_at TIMESTAMP,
archived_by UUID,
archived_reason TEXT,
sync_hash VARCHAR(64),
entity_hash VARCHAR(64),
last_sync_at TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Companies (entreprises)

```

-- Schéma: companies
CREATE SCHEMA companies;

-- Entreprises
CREATE TABLE companies.companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  website VARCHAR(255),
  industry VARCHAR(100),
  size VARCHAR(50),
  location VARCHAR(255),
  description TEXT,
  logo_url VARCHAR(500),
  sync_hash VARCHAR(64),
  entity_hash VARCHAR(64),
  last_sync_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```



Migration depuis v4.0

Étapes de migration

1. **Sauvegarde** de la base existante

2. **Application des migrations** : `npx prisma migrate dev`
3. **Mise à jour des services** pour utiliser les nouvelles relations
4. **Test des fonctionnalités** ajoutées

Script de migration

```
-- Migration des données existantes
-- Ajout des nouvelles colonnes et relations
-- Mise à jour des indexes
-- Validation de l'intégrité des données
```



Ressources

- [Architecture complète](#) - Vue technique
- [Services](#) - Détail des microservices
- [API Reference](#) - Documentation API
- [Prisma Schema](#) - Schéma source

Version: 4.1 - Base de données étendue **Dernière mise à jour:** Octobre 2025